How Matrix Size Drives Runtime and Memory of a Basic Triple-Loop Matrix Multiplication in Python, Java and C

Big Data – Individual Assignment 1: Basic Matrix Multiplication in Different Languages

Aimar Daniel Alejandro Santana
Grado en Ciencia e Ingeniería de Datos, Universidad de Las Palmas de Gran Canaria

Repository: `https://github.com/YOUR-USER/BigData-Assignment01-MatMul`

October 2026

Abstract

We measure how the matrix size n affects the runtime and the peak memory of one and the same basic $O(n^3)$ triple-loop (i-j-k) dense matrix multiplication written in C, Java and Python. The three versions read identical binary64 input files and produce bit-identical results. They were timed on the same 2-vCPU Intel Xeon virtual machine, with only the kernel inside the timer, 5–10 repetitions per configuration (median and IQR reported), and a declared budget of 200 s per multiplication. Every language grows at least cubically, but the cost per inner iteration is not constant. It rises with n, by up to $\approx 8\times$ for C, $\approx 14\times$ for Java and $\approx 2.5\times$ for Python, because the stride-n column access to B steps out of the caches. C and Java stay within a factor of ≈ 0.7–2.6 of each other. Neither is uniformly faster: Java is ahead at $n = 512$ and 768, and C at $n \geq 1024$. CPython is 41–$104\times$ slower than C. The largest feasible sizes are $n = 2048$ for Java, $n = 1536$ for C (its $n = 2048$ run exceeded the budget in the main experiment but took 108 s in a separate probe) and $n = 1024$ for Python. Peak memory grows as n^2 on top of a fixed runtime cost of about 2 MiB (C), 41 MiB (Java) and 11 MiB (Python). These conclusions apply only to the tested implementations, settings and machine.

1 Introduction and objective

Dense matrix multiplication is the reference kernel of numerical computing and a natural starting point for a course on scalable data processing: it is simple to state, its arithmetic cost is known exactly, and its performance depends strongly on how the implementation interacts with the processor and the memory hierarchy. Before optimised, sparse, parallel and distributed variants are studied in later assignments, this study establishes a validated, reproducible *baseline* for the same basic algorithm in three languages with very different execution models: an ahead-of-time compiled language (C), a JIT-compiled managed language (Java) and an interpreted dynamic language (CPython).

Research question. *How does the matrix size n affect the runtime and the peak memory of the same basic $O(n^3)$ triple-loop multiplication implemented in Python, Java and C, when inputs, algorithm, loop order, numeric type, timing boundary and machine are held fixed?*

Factor, outcomes and comparison. The factor varied is n (side of an $n \times n$ matrix); the categorical factor is the implementation language. The outcomes are (i) time per multiplication of the kernel and (ii) peak resident memory of the process.

Hypotheses (predictions, stated before running the experiments).

- H1 Runtime grows approximately as n^3 in all three languages, but the constant per inner iteration is not constant: it should increase for large n because the column-wise access to B stops fitting in cache.
- H2 C and Java (after JIT warm-up) are within a small factor of each other, while CPython is one to two orders of magnitude slower because each multiply-add is executed by the bytecode interpreter on boxed float objects.
- H3 Peak memory grows as n^2; Java adds a large fixed runtime cost, and Python needs several times more bytes per element than the 8 bytes of a raw `double`.

2 Background

Algorithm. For $C = AB$ with $A, B \in \mathbb{R}^{n \times n}$, the definition $c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$ gives the classical triple loop with exactly n^3 multiplications and n^3 additions ($2n^3$ floating-point operations) [1]. The six loop orders perform the same arithmetic but access memory differently. In the *i-j-k* order used here, the innermost loop reads row i of A contiguously (stride 1) and column j of B with stride n elements, which defeats spatial locality and makes the kernel memory-bound once B no longer fits in the caches [2, 3]. When the stride is a large power of two, consecutive elements of a column map to a few cache sets and evict each other (conflict misses) [4].

Execution models. C is compiled ahead of time to machine code. Java bytecode is first interpreted and then compiled by the HotSpot JIT compiler once methods or loops become hot, so early executions are slower and warm-up runs must be separated from measured runs [5]. CPython interprets bytecode; every `float` is a heap object (24 bytes on 64-bit CPython, measured with `sys.getsizeof`) referenced by an 8-byte pointer from its list, and every arithmetic operation involves type dispatch and the allocation of a new result object [6].

Floating-point accuracy. A length-n dot product in binary64 satisfies $|\hat{c}_{ij} - c_{ij}| \leq \gamma_n (|A||B|)_{ij}$ with $\gamma_n = nu/(1-nu)$ and $u = 2^{-53}$ [7]. This bound is used as the tolerance for validation. Since Java 17 all floating-point arithmetic is strict IEEE 754 [8]; with GCC `-O2` on the baseline x86-64 target no fused multiply-add is emitted and no reassociation is allowed (no `-ffast-math`); CPython performs one IEEE operation per bytecode. The three kernels therefore execute the same sequence of rounded operations, which makes a bitwise comparison of their outputs a strong correctness check.

Benchmark reporting. Following common recommendations for performance studies [9, 5], we keep every raw sample, summarise with the median and the interquartile range (IQR), report the number of repetitions, and describe the timer boundary.

3 Methodology

3.1 Implementations and storage layouts

All three programs implement the same kernel, with the same loop order (i-j-k), the same local scalar accumulator and the same numeric type (IEEE 754 binary64):

```
for i in 0..n-1: for j in 0..n-1:
    s = 0.0
    for k in 0..n-1: s += A[i][k] * B[k][j]
    C[i][j] = s
```

No library call replaces the loops. Unavoidable language-specific differences are the storage layouts (Table 1). Source files: `src/c/matmul.c`, `src/java/MatMul.java`, `src/python/matmul.py`.

Table 1: Implementations and fixed settings.

Language	Storage layout	Toolchain and fixed settings
C	One contiguous row-major block of n^2 `double` per matrix plus an array of n row pointers (`A[i][k]` syntax).	GCC 13.3.0, `gcc -O2 -std=c11 -Wall -Wextra` (no `-march`, no `-ffast-math`).
Java	`double[][]`: each row is a separate contiguous `double[]` heap object (16-byte header); rows are not guaranteed to be adjacent.	OpenJDK 21.0.12 HotSpot Server VM, `javac -release 21`; `java -Xms64m -Xmx4g -XX:+UseSerialGC`; default tiered JIT; 3 warm-up runs in the same JVM.
Python	List of lists; each element is a boxed `float` object (24 B) referenced by an 8 B pointer.	CPython 3.13.16 (no JIT enabled, standard GIL build); pure-Python loops; NumPy not used in the kernel; 1 warm-up run.

3.2 Inputs

All implementations read the *same* binary files, so the logical inputs are identical across languages. The matrices are generated once by `scripts/generate_inputs.py` with NumPy's PCG64 generator [10] (seed 20261012 for A, 20261013 for B), uniformly distributed in $[-1, 1)$, stored as raw little-endian row-major binary64. A manifest with the SHA-256 hash of every file is saved with the data.

3.3 Environment

Table 2: Execution environment (full output of `scripts/collect_environment.sh` in `results/environment.txt`).

CPU	Intel Xeon Processor @ 2.80 GHz (Cascade Lake family 6 model 85, AVX-512 capable), KVM guest
Cores / logical CPUs	2 vCPUs: 1 socket, 2 cores, 1 thread per core; no affinity pinning
Caches (as reported)	L1d 32 KiB/core, L2 1 MiB/core, L3 33 MiB shared
RAM	7.8 GiB (8 421 236 736 bytes), no swap; transparent huge pages: `madvise`
OS	Ubuntu 24.04.5 LTS, Linux 6.18 (cloud virtual machine, shared physical host)
Software	gcc 13.3.0; OpenJDK 21.0.12 (HotSpot); CPython 3.13.16; NumPy 2.5.3 (inputs/reference only)
Threads	All kernels single-threaded; JVM internal threads (JIT, 1 serial GC thread) not controlled

Because the machine is a virtual machine on a shared host, other tenants can disturb caches and CPU time. Every configuration is measured on the same machine, so the comparison between languages is controlled. Absolute times, however, apply only to this environment.

3.4 Correctness checks

Before benchmarking, `tests/validate.py` runs every implementation on: a hand-computed 2×2 and 3×3 product, $IR = R$, $RI = R$, $0R = 0$, $n = 1$ (all required to be *exactly* equal), and random matrices with $n \in \{7, 33, 100, 256\}$, compared against NumPy (used only as a reference, not as an implementation) with the componentwise tolerance $|C - C_{\text{ref}}| \leq 2\gamma_n(|A||B|)$; the factor 2 accounts for the rounding error of the reference itself. In addition the outputs of the three languages are compared for *bitwise* equality. During every benchmark run a checksum of C is computed after the timer is stopped, so the result is always consumed and cannot be optimised away.

3.5 Measurement protocol

- **Timer boundary: kernel only.** Input loading and allocation/zeroing of C happen before the timer starts; the checksum is computed after it stops. Monotonic timers: `clock_gettime(CLOCK_MONOTONIC)` (C), `System.nanoTime()` (Java), `time.perf_counter_ns()` (Python).
- **Sizes:** $n \in \{64, 128, 256, 384, 512, 768, 1024, 1536, 2048\}$ (powers of two plus intermediate values to expose trends).
- **Repetitions:** one fresh process per (language, n); 10 measured samples for $n \leq 512$ and 5 for $n > 512$ (justified by cost: one Python run at $n = 1024$ takes about 2.5 minutes). Warm-up runs, recorded but excluded: C 1, Java 3, Python 1. For $n = 64$ and $n = 128$ each sample times a batch of 20 and 5 consecutive multiplications respectively and the time per multiplication is reported.
- **Budgets (declared in advance):** at most 200 s per single multiplication and 4 GB peak RSS. If a configuration exceeds the budget it is killed, recorded as TIMEOUT, and larger sizes for that language are not attempted.
- **Memory metric:** peak resident set size of the benchmark process (`VmHWM`), read by each program from `/proc/self/status` just before exiting [11]. It includes the runtime (interpreter/JVM), the three matrices and any temporaries; it is *not* the size of the arrays alone. A baseline run per language that loads the $n = 64$ inputs but performs no multiplication measures the fixed runtime cost. We also report the analytical raw array size $3 \cdot 8n^2$ bytes. For Java the JVM's own heap statistics are logged but are a different quantity and are not compared with RSS.
- **Execution order and noise:** configurations run sequentially (increasing n; C, Java, Python) with no other user workload. The kernels are single-threaded; the JVM still runs internal threads (JIT compiler, one serial GC thread), which may use the second vCPU.
- **Summary statistics:** median and IQR (with min/max) of measured samples; every raw sample is kept in `results/raw_samples.csv`.

4 Results

4.1 Correctness

All 40 checks passed (`results/validation.csv`). The six exact cases gave exactly the expected matrices in all three languages. In the random cases the largest error with respect to NumPy was 1.1×10^{-14} at $n = 256$, which is at most 9 % of the allowed bound $2\gamma_n(|A||B|)$. The outputs of C, Java and Python were *bitwise identical* in every case. This confirms that the three kernels perform the same sequence of IEEE operations. It also means that differences in runtime cannot come from differences in arithmetic.

4.2 Runtime

Table 3 reports every configuration and Figure 1 plots the medians on log–log axes. All configurations completed within the budget except Python at $n = 1536$ and C at $n = 2048$, which exceeded 200 s in the first (warm-up) run. Following the stop rule, Python at $n = 2048$ was not attempted. The largest tested feasible sizes are therefore $n = 2048$ (Java), $n = 1536$ (C) and $n = 1024$ (Python). The largest attempted size was 2048 for C and Java and 1536 for Python. Relative variability was mostly below 12 % of the median. The exception is Python at $n = 256$ (20 %), which we attribute to interference on the shared host.

Table 3: Time per multiplication (kernel only). Reps = measured samples (warm-up excluded); "$\times b$" = each sample timed b consecutive multiplications. IQR is also given as a percentage of the median. ns/iter = median$/n^3$. MFLOP/s = $2n^3$/median.

Language	n	Reps	Median	IQR (% of median)	Min	Max	ns/iter	MFLOP/s
C	64	10 (×20)	260.9 µs	15.8 µs (6.1%)	250.8 µs	285.7 µs	1.00	2010
C	128	10 (×5)	2.39 ms	17.1 µs (0.7%)	2.38 ms	2.42 ms	1.14	1753
C	256	10	20.89 ms	586.4 µs (2.8%)	20.45 ms	21.63 ms	1.25	1606
C	384	10	76.17 ms	1.68 ms (2.2%)	74.57 ms	78.09 ms	1.35	1487
C	512	10	362.28 ms	42.48 ms (11.7%)	329.41 ms	420.15 ms	2.70	741
C	768	5	1.23 s	47.19 ms (3.8%)	1.12 s	1.29 s	2.72	736
C	1024	5	3.71 s	335.66 ms (9.1%)	3.46 s	4.48 s	3.45	579
C	1536	5	28.03 s	1.47 s (5.2%)	26.56 s	28.49 s	7.74	259
C	2048	–	*timeout* (budget 200 s per multiplication)					
Java	64	10 (×20)	282.6 µs	5.5 µs (1.9%)	276.5 µs	288.2 µs	1.08	1855
Java	128	10 (×5)	2.47 ms	112.2 µs (4.5%)	2.44 ms	3.06 ms	1.18	1695
Java	256	10	24.62 ms	2.51 ms (10.2%)	21.70 ms	31.12 ms	1.47	1363
Java	384	10	100.23 ms	5.83 ms (5.8%)	94.04 ms	104.97 ms	1.77	1130
Java	512	10	267.84 ms	29.05 ms (10.8%)	238.21 ms	348.50 ms	2.00	1002
Java	768	5	874.30 ms	100.61 ms (11.5%)	842.59 ms	1.05 s	1.93	1036
Java	1024	5	9.68 s	81.77 ms (0.8%)	9.59 s	9.90 s	9.01	222
Java	1536	5	51.26 s	1.04 s (2.0%)	50.00 s	51.55 s	14.14	141
Java	2048	5	125.84 s	997.69 ms (0.8%)	124.75 s	127.38 s	14.65	137
Python	64	10 (×20)	15.55 ms	1.79 ms (11.5%)	15.00 ms	19.24 ms	59.31	34
Python	128	10 (×5)	143.61 ms	7.05 ms (4.9%)	132.27 ms	154.70 ms	68.48	29
Python	256	10	1.65 s	337.37 ms (20.4%)	1.30 s	2.08 s	98.35	20
Python	384	10	7.91 s	260.58 ms (3.3%)	7.51 s	8.11 s	139.64	14
Python	512	10	19.84 s	475.10 ms (2.4%)	18.99 s	20.90 s	147.78	14
Python	768	5	74.78 s	1.90 s (2.5%)	73.91 s	76.95 s	165.07	12
Python	1024	5	152.94 s	552.96 ms (0.4%)	152.38 s	153.59 s	142.44	14
Python	1536	–	*timeout* (budget 200 s per multiplication)					
Python	2048	–	*not attempted* (budget 200 s per multiplication)					

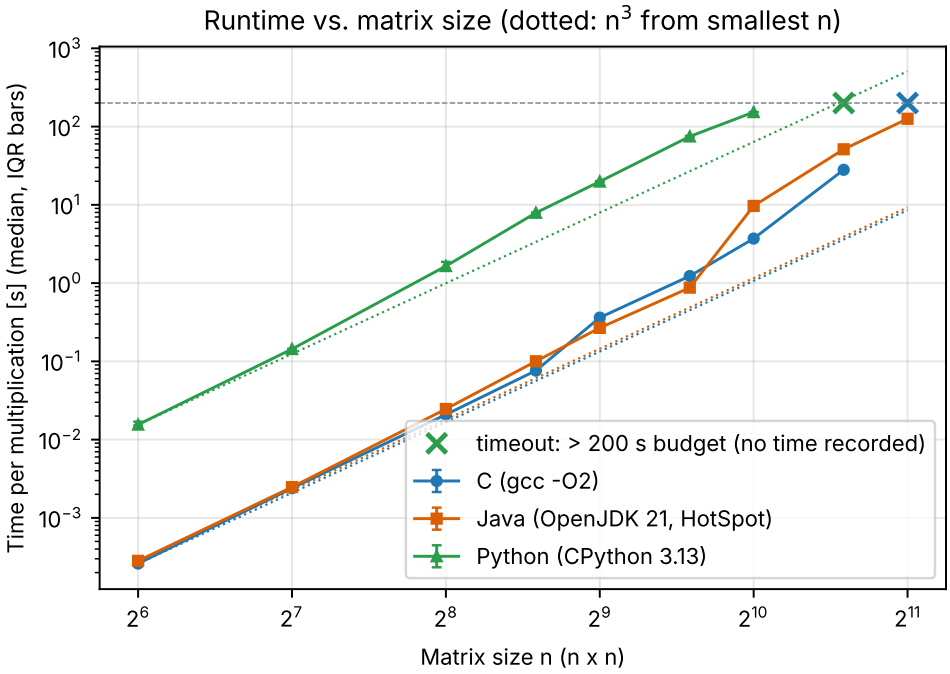


Figure 1: Median time per multiplication against n (log–log). Error bars show the IQR (mostly smaller than the markers). Dotted lines extrapolate an exact n^3 law from the smallest size of each language. Crosses mark configurations killed at the 200 s budget.

Growth versus the cubic model. Table 4 fits $\log t = a + b \log n$ for $n \geq 256$. Python grows close to cubically ($b = 3.27$). C ($b = 3.97$) and Java ($b = 4.29$) grow clearly faster than n^3 over the sizes tested. Figure 2 explains why: if growth were exactly cubic, the time per inner iteration (median$/n^3$) would be flat. Instead it increases in steps. For C it goes from 1.0 ns ($n = 64$) to 2.7 ns ($n = 512$–768), 3.5 ns ($n = 1024$) and 7.7 ns ($n = 1536$). Java follows a similar path but jumps between $n = 768$ (1.9 ns) and $n = 1024$ (9.0 ns), reaching 14.7 ns at $n = 2048$. Python rises from 59 ns to 140–165 ns.

Table 4: Least-squares slope of log(median time) against $\log n$ for $n \geq 256$ (slope 3 = cubic).

Language	Sizes	Points	Slope b	R^2
C	256–1536	6	3.97	0.9938
Java	256–2048	7	4.29	0.9834
Python	256–1024	5	3.27	0.9951

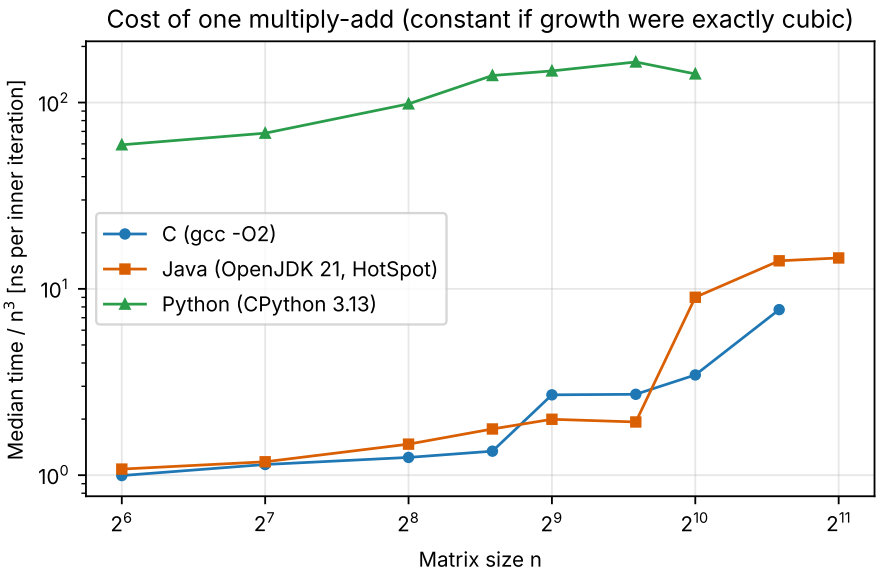


Figure 2: Median time divided by n^3: the cost of one multiply-add of the inner loop.

Relative performance. Table 5 and Figure 3 give the ratio of medians with respect to C. Java is within 8–32 % of C for $n \leq 384$ and faster than C at $n = 512$ and 768 (ratio 0.74 and 0.71). It is 2.6× and 1.8× slower at $n = 1024$ and 1536. Python is 41–104× slower than C.

Table 5: Median time ratios.

n	Java/C	Python/C
64	1.08	59.6
128	1.03	60.0
256	1.18	79.0
384	1.32	103.8
512	0.74	54.8
768	0.71	60.8
1024	2.61	41.3
1536	1.83	–
2048	–	–

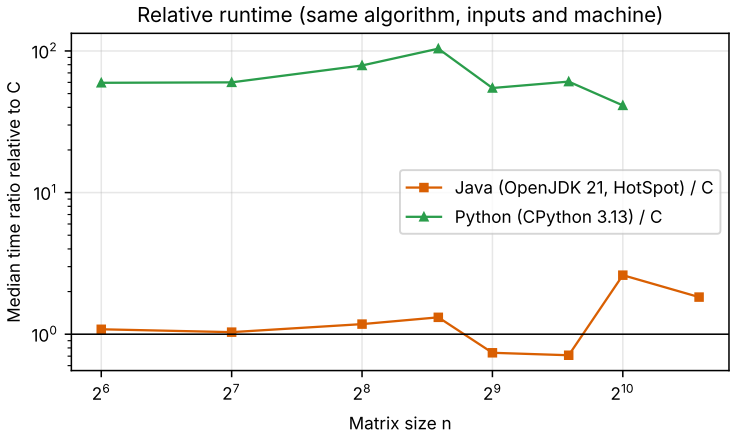


Figure 3: Ratios of median time relative to C.

Java warm-up. Figure 4 shows each run inside one JVM, divided by the median of the measured runs. The first execution at $n = 128$ (a batch of five multiplications) takes 1.7× the steady-state time, because the code is still interpreted or only partly compiled. From the second run onwards the times fall within normal noise. Three warm-up runs are therefore enough, and excluding them avoids biasing Java's results.

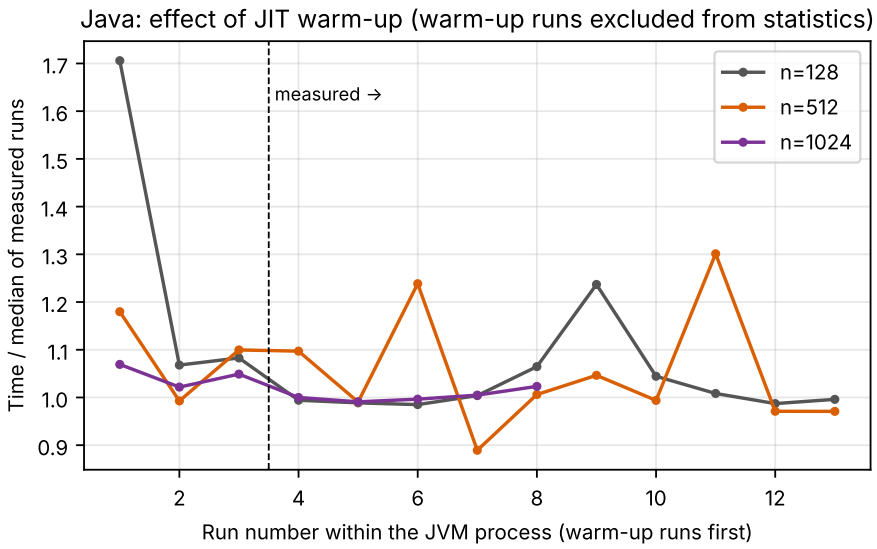


Figure 4: Java: time of every run inside one JVM process divided by the median of the measured runs. The first three runs are warm-up and are excluded from the statistics.

4.3 Memory

Table 6 and Figure 5 report peak RSS (VmHWM). The baseline row shows the fixed cost of each runtime after loading the $n = 64$ inputs without multiplying. The cost on top of that baseline grows as n^2:

- **C**: peak RSS is close to the raw array size plus 1.7 MiB (e.g. 25.7 MiB against 24.0 MiB of arrays at $n = 1024$).
- **Java**: about 41 MiB of fixed JVM cost plus heap. At $n = 2048$ the 205 MiB includes the 96 MiB of arrays, the 32 MiB transient `byte[]` used when reading each input file, and the space the garbage-collected heap has committed but not yet released.
- **Python**: grows fastest. At $n = 1024$ the 131 MiB is about 5.5× the raw arrays. Subtracting the baseline gives ≈ 40 bytes per element. This matches a 24-byte `float` object plus an 8-byte list pointer per element, plus list headers and the transient `array` buffer used while loading the inputs.

These numbers measure different things (process RSS, not live data), so they should be read as the memory footprint of each program, not as the size of the matrices.

Table 6: Peak RSS [MiB] and raw array size $3 \cdot 8n^2$ bytes [MiB].

n	Arrays	C	Java	Python
baseline	–	1.7	41.4	10.8
64	0.1	1.8	42.7	11.1
128	0.4	2.1	43.3	12.5
256	1.5	3.2	45.6	18.2
384	3.4	5.1	49.5	27.7
512	6.0	7.7	55.2	40.8
768	13.5	15.2	74.9	78.6
1024	24.0	25.7	93.9	131.3
1536	54.0	55.8	133.3	–
2048	96.0	–	205.4	–

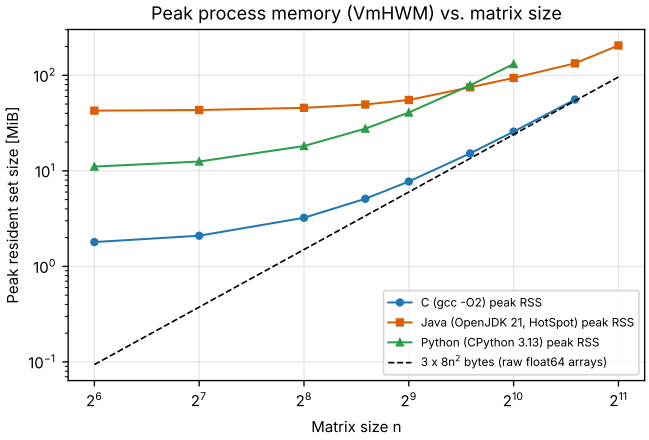


Figure 5: Peak RSS against n (log–log); the dashed line is the raw size of the three float64 arrays.

4.4 Supplementary experiments (separate from the controlled comparison)

Two unexpected observations from the main study were examined with additional runs. These used the same kernel, flags, input generator and timer boundary, under their own declared settings (`config/supplementary_*.json`).

A. Sizes around powers of two. C shows a sharp step at $n = 512$ that Java does not. We therefore measured $n \in \{500, 511, 512, 513, 1000, 1023, 1024, 1025\}$ (Table 7, Figure 6). For C, $n = 512$ costs 2.76 ns per iteration, compared with 1.47–1.53 ns at 500, 511 and 513. Java shows no such step (1.82–1.94 ns). Around $n = 1000$ there is no power-of-two effect in either language: all C sizes lie at 3.4–4.9 ns and all Java sizes at 9.4–10.1 ns.

Table 7: Supplementary A: median ns per inner iteration.

n	C	Java
500	1.47	1.90
511	1.47	1.94
512	2.76	1.82
513	1.53	1.88
1000	3.76	9.37
1023	4.95	10.15
1024	3.41	9.45
1025	3.38	10.11

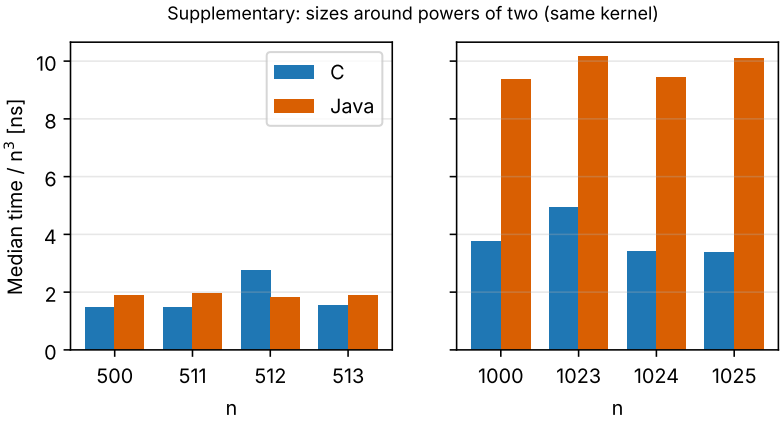


Figure 6: Supplementary A: cost per inner iteration at and around powers of two.

B. The C time-out at $n = 2048$. Fresh processes for $n = 2047$ and $n = 2048$ were run with an extended 900 s budget and 3 measured runs (Table 8). In this probe C completed $n = 2048$ in a median of 107.6 s, well inside the original budget. In the main run it had not finished a single multiplication in 200 s. C at $n = 2047$ took 95.3 s. Java took ≈ 130 s at both sizes, close to its main-study value (125.8 s). The main-study time-out is kept as a result. It shows that the time of one C process at $n = 2048$ can vary by more than a factor of two between processes. Within each process the runs were very stable.

Table 8: Supplementary B: extended-budget probe (900 s), 3 measured runs per configuration.

Language	n	Reps	Median [s]	Range [s]	ns/iter
C	2047	3	95.3	94.9–95.4	11.11
C	2048	3	107.6	107.1–107.7	12.52
Java	2047	3	129.9	129.6–132.0	15.15
Java	2048	3	129.6	129.3–129.8	15.09

5 Discussion

Answer to the research question. On this machine, runtime grows with n at least as fast as the cubic operation count for all three implementations (H1 confirmed). Peak memory grows as n^2 above a language-specific fixed cost (H3 confirmed). Between languages, the gap is large and stable for Python (≈ 40–$100\times$ slower than C). For C and Java the relation depends on n: neither is faster at every size, so H2 holds only in part.

Interpreter effects (Python). With bitwise-identical arithmetic, Python's cost of 59–165 ns per inner iteration, against 1–8 ns in C, comes from executing the inner loop as bytecode. Each `s += A[i][k]*B[k][j]` performs several list subscriptions, dynamic type checks and the allocation of new `float` objects [6]. This per-operation overhead is so large that memory effects matter less, which is why Python's growth exponent (3.27) is the closest to the cubic model. Its per-iteration cost still rises by $\approx 2.5\times$ between $n = 64$ and $n = 768$. A plausible explanation is that the boxed `float` objects of a column of B are scattered in memory, so they stop fitting in cache. We did not measure cache misses, so this remains a hypothesis.

Runtime effects (Java). After warm-up, HotSpot's JIT-compiled loop is competitive with `gcc -O2` for $n \leq 768$, as long as the working set stays in cache. Excluding the warm-up runs was essential: including them would add up to 70 % to the first sample at $n = 128$ [5]. The JVM also has the largest fixed memory cost.

Memory-layout effects (C and Java). In i-j-k order the inner loop walks down a column of B with a stride of $8n$ bytes. A cache line brings in $B[k][j..j+7]$, so lines can be reused by the next seven values of j, but only if the n lines of a column stay in cache. As n grows, the column and then all of B stop fitting in L1, L2 and L3. Each level that is exceeded raises the cost per iteration [2, 3]. This produces the steps in Figure 2 and the slopes above 3. Supplementary experiment A supports a memory-layout explanation for one specific step. In C every row of B starts exactly $8n$ bytes after the previous one. At $n = 512$ this is 4096 bytes, so all elements of a column map to the same L1 cache set and evict each other (conflict misses) [4]. Neighbouring sizes (511, 513) do not show the step. In Java each row is a separate array with a 16-byte header, so consecutive rows are not exactly a power of two apart. Java shows no step at 512, which is why it beats C there. Above $n \approx 1000$, Java pays more per iteration than C (9–15 ns against 3.4–4.9 ns at $n \approx 1000$). This is not a power-of-two effect (supplementary A). Candidate explanations include:

- the extra instructions per iteration (row-reference load, bounds check), which reduce the number of cache misses the CPU can overlap;
- address-translation (TLB) misses, because at this size each access to B touches a different 4 KiB page;
- the heap placement of the row arrays.

Hardware performance counters are not available in this virtual machine, so we cannot decide between them. This question remains open.

Limitations.

- The machine is a 2-vCPU cloud virtual machine on a shared host. Absolute times, and in particular the C $n = 2048$ result (time-out in one process, 108 s in another), may be affected by other tenants and by the physical pages the operating system assigns. We could not separate these causes.
- No performance counters were available, so the explanations based on caches and the TLB are hypotheses consistent with the data, not measurements.
- Large sizes use only 5 repetitions (3 in the probe) because of their cost. Each configuration runs in one process, so variation between processes is captured only by the supplementary probe.
- Only one compiler level (`-O2`), one JVM configuration and one Python implementation were tested. Other settings (e.g. `-O3 -march=native`, PyPy, another loop order) are language-specific optimisations outside this controlled baseline.
- Peak RSS includes transient buffers used to load the inputs and, for Java, heap space that is committed but not live. It measures each program's footprint, not the matrices alone.

6 Conclusions

Within the tested scope (one 2-vCPU Intel Xeon VM; gcc 13.3 `-O2`, OpenJDK 21 HotSpot after 3 warm-up runs, CPython 3.13; float64 inputs uniform in $[-1, 1)$; kernel-only timing), this study establishes four findings:

1. The three straightforward triple-loop implementations are correct and produce bit-identical results.
2. Their runtime grows at least cubically with n. Beyond the n^3 operation count, the memory hierarchy adds a cost per iteration that grows with n: up to $\approx 8\times$ in C, $\approx 14\times$ in Java and $\approx 2.5\times$ in Python between the smallest and largest feasible sizes.
3. CPython is about 40–100 times slower than C. C and Java are close for small sizes, and which one is faster depends on n and on the memory layout (power-of-two strides hurt C at $n = 512$; Java is $\approx$ 2–2.6$\times$ slower from $n \approx 1000$).
4. Peak memory grows as n^2 above fixed runtime costs of about 2, 41 and 11 MiB, and Python needs about 40 bytes per matrix element.

The largest feasible sizes under the 200 s budget were 2048 (Java), 1536 (C) and 1024 (Python). These results do not show that any language is universally faster. Two questions remain open: the cause of Java's slowdown above $n \approx 1000$, and the variability of C at $n = 2048$. Answering them requires hardware counters or a dedicated machine. The memory-access pattern identified here is the main target of the optimisations (loop reordering, blocking) studied in Assignment 2.

AI-use disclosure

In line with the course's generative-AI policy, the AI assistant Claude (Anthropic) was used to help plan the experimental protocol, draft and debug the benchmark scripts and analysis code, and revise the English of this report. All measurements were produced by executing the code in the repository; no number in this report was generated by the AI tool. The author reviewed the code, re-ran the validation, checked that every cited source exists and supports the statement, and is responsible for the conclusions.

Reproducibility materials

Everything needed to reproduce this study is in the repository `https://github.com/YOUR-USER/BigData-Assignment01-MatMul` (and in `Assignment01_AlejandroSantana_Materials.zip`): source code, `README.md` with exact build/run commands, `config/*.json` (sizes, repetitions, warm-ups, budgets, JVM flags), the input generator and manifest, raw measurements (`results/raw_samples.csv`, `results/runs.csv`, process logs), the validation output, the environment description, and the scripts that produce every table and figure (`scripts/analyze*.py`). The command `./run_all.sh` rebuilds, re-validates, re-measures and regenerates the report.

References

[1] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.

[2] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., Nov. 2007. [Online]. Available: `https://people.freebsd.org/~lstewart/articles/cpumemory.pdf`

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*, 1991, pp. 63–74.

[5] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76.

[6] Python Software Foundation, "The Python Language Reference, §3: Data model," Python 3.13 documentation. [Online]. Available: `https://docs.python.org/3.13/reference/datamodel.html`

[7] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[8] J. D. Darcy, "JEP 306: Restore always-strict floating-point semantics," OpenJDK, 2021. [Online]. Available: `https://openjdk.org/jeps/306`

[9] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC'15)*, 2015, pp. 73:1–73:12.

[10] M. E. O'Neill, "PCG: A family of simple fast space-efficient statistically good algorithms for random number generation," Harvey Mudd College, Claremont, CA, USA, Tech. Rep. HMC-CS-2014-0905, 2014.

[11] Linux man-pages project, "proc_pid_status(5) — memory usage and status information," [Online]. Available: `https://man7.org/linux/man-pages/man5/proc_pid_status.5.html`